



Fiche TP 4 : Sur l'Héritage et les Exceptions

OBJECTIFS :

- Identifier une situation d'exception ; interpréter un héritage ; interpréter une interface JAVA
- Déclarer et utiliser une classe d'Exception ; Hériter une classe d'une autre ; Déclarer une interface et d'en hériter des classes

EXERCICE 1 : Déclenchement et traitement d'exceptions

Dans cette activité, nous allons traiter un exemple d'utilisation des exceptions. Pour cela, nous allons définir une classe appelée *EntiersNaturels*, représentant les entiers naturels (positifs ou nul). Pour avoir un objet de cette classe, il faut l'instancier en fournissant au constructeur un entier (positif ou nul). Quand l'entier fourni au constructeur est négatif, une exception doit être déclenchée, permettant d'afficher un message d'erreur et en interrompant l'exécution. Nous déclarerons la classe *ErreurConstructeur* pour représenter cette exception.

Ici, la classe *ErreurConstructeur* peut être réduite à sa plus simple expression, à savoir ne comporter ni champs ni méthodes, mais elle doit nécessairement hériter de la classe *Exception* de JAVA :

```
public class ErreurConstructeur extends Exception{
```

La définition de la classe *EntiersNaturels* pourrait se présenter ainsi, avec son constructeur qui doit déclencher une exception de type *ErreurConstructeur* lorsque la valeur reçue est négative. :

```
public class EntiersNaturels {  
    public EntiersNaturels (int n) throws ErreurConstructeur  
    { if (n<0) throw new ErreurConstructeur() ;  
      This.n = n ;  
    }  
    public int getN () { return n ; }  
    private int n ;  
}
```

Voici un programme d'utilisation dans lequel nous traitons l'exception *ErreurConstructeur* en incluant les instructions concernées dans un bloc **try** que nous faisons suivre d'un gestionnaire introduit par **catch(*ErreurConstructeur e*)**. Comme demandé, nous y affichons un message (***** erreur construction *****) et nous mettons fin à l'exécution par l'appel de *System.exit* :

```

public class TestEntiersNaturels {
    public static void main (String args[])
    { try
    { EntiersNaturels n1 = new EntiersNaturels(20) ;
    System.out.println ("n1 = " + n1.getN() ) ;
    EntiersNaturels n2 = new EntiersNaturels(-12) ;
    System.out.println ("n2 = " + n2.getN() ) ;
    }
    catch (ErreurConstructeur e)
    { System.out.println ("*** erreur construction ***") ;
    System.exit (-1) ;
    }
    }
}

```

Travail à faire : Créer un Package appelé entierNaturel et déclarez y les trois classes précédentes : *EntierNaturel*, *ErreurConstructeur*, *TestEntiersNaturels*. Exécuter la classe *TestEntiersNaturels* et Faites une capture de l'écran d'exécution que vous déposez sur la plateforme.

EXERCICE 2 : Déclenchement et traitement d'exceptions – Bis

Reprendre l'**exercice 1** en faisant de sorte que le programme demande successivement des entiers naturels et les lire jusqu'à ce que l'erreur survienne : l'instruction *while(true) { bloc d'instructions}*, permet de répéter indéfiniment le *bloc d'instructions*.

La classe Scanner permet de lire un flux de données formatées. Pour l'utiliser afin de pouvoir lire des données à travers votre programme, il faut instancier un objet de cette classe :

```
Scanner entree = new Scanner(System.in);
```

La classe *TestEntiersNaturels* peut être écrite de la sorte :

```

public class TestEntiersNaturels {
    public static void main (String args[])
    {
    int x;
    Scanner entree = new Scanner(System.in);
    try
    { while(true){System.out.println("donner un entier");
    x=entree.nextInt();
    EntiersNaturels n1 = new EntiersNaturels(x) ;
    System.out.println ("n1 = " + n1.getN() ) ;
    }
    catch (ErreurConstructeur e)
    { System.out.println ("*** erreur construction ***") ;
    System.exit (-1) ;
    }
    }
}

```

EXERCICE 3 : Définition de classes dérivées¹

On dispose de la classe suivante :

```
class Point  
{ public void initialise (int x, int y) { this.x = x ; this.y = y ; }  
public void deplace (int dx, int dy) { x += dx ; y += dy ; }  
public int getX() { return x ; }  
public int getY() { return y ; }  
private int x, y ;  
}
```

Nous souhaitons réaliser une classe *PointA*, dérivée de *Point* disposant d'une méthode *affiche* affichant (en fenêtre console) les coordonnées d'un point. Ecrire un petit programme utilisant les deux classes *Point* et *PointA*.

Pour cela, il suffit de définir la classe *PointA* en utilisant le mot clé *extends*. La méthode *affiche*, comme toute méthode d'une classe dérivée a accès à tous les membres publics de la classe de base, donc en particulier à *getX* et *getY*.

```
class PointA extends Point  
{ void affiche()  
{ System.out.println ("Coordonnees : " + getX() + " " + getY());  
}  
}
```

On peut alors créer des objets de type *PointA* et leur appliquer aussi bien les méthodes publiques de *PointA* que celles de *Point* comme dans ce programme accompagné d'un exemple d'exécution:

```
public class TsPointA  
{ public static void main (String args[])  
{ Point p = new Point () ;  
  p.initialise (2, 5) ;  
  System.out.println ("Coordonnees : " + p.getX() + " " + p.getY() ) ;  
  PointA pa = new PointA () ;  
  pa.initialise (1, 8) ; // on utilise la methode initialise de Point  
  pa.affiche() ;      // et la methode affiche de PointA  
}  
}
```

EXERCICE 4 : Définition de classes dérivées – Bis²

On dispose de la classe suivante :

```
class Point  
{ public void setPoint (int x, int y) { this.x = x ; this.y = y ; }  
public void deplace (int dx, int dy) { x += dx ; y += dy ; }
```

^{1,2} Exercice tiré du livre : « Exercices en JAVA », Claude Delannoy, 2^{ème} édition JAVA 5.0, EYROLLES

```

    public void affCoord ()
    { System.out.println ("Coordonnees : " + x + " " + y) ;
    }
    private int x, y ;
}

```

Travail à faire : Réaliser une classe *PointNom*, dérivée de *Point* permettant de manipuler des points définis par deux coordonnées (*int*) et un nom (caractère). On y prévoira les méthodes suivantes :

- *setPointNom* pour définir les coordonnées et le nom d'un objet de type *PointNom*,
- *setNom* pour définir seulement le nom d'un tel objet,
- *affCoordNom* pour afficher les coordonnées et le nom d'un objet de type *PointNom*.

Écrire un petit programme utilisant la classe *PointNom*.

Consignes :

- Définir la classe dérivée *PointNom* en utilisant le mot clé *extends* :

class PointNom extends Point

- Dans cette classe *PointNom*, introduire un champ (de préférence privé) destiné à contenir le nom du point : ***private char nom ;***

- La méthode *setNom* est triviale. Compte tenu de l'encapsulation des données de *Point*, nos deux autres méthodes doivent absolument recourir aux méthodes publiques de *Point*.